DESENVOLVIMENTO ASSISTIDO POR IA · BLOCO 5

Revisão, segurança e dívida do código assistido

5

Sábado, 24/10/2026 · 9h às 12h

Ferramenta: Claude Code · modo manual por padrão

Pós-Graduação em Engenharia de Software e IA · CESAR School

CESAR

ABERTURA

Vocabulário do bloco

code review	revisão de código antes do merge
SAST	análise estática do código-fonte
SCA	análise das dependências
IDOR	acesso a objeto de outro usuário pelo ID
SQL injection	entrada do usuário vira comando SQL
path traversal	nome de arquivo que sai do diretório
supply chain	cadeia de suprimentos de software
slopsquatting	registrar pacote com nome alucinado
lockfile	arquivo que fixa as versões exatas
automation bias	viés de automação: confiar demais na máquina

Você aprovou um PR gerado por IA sem rodar o código? Quantas vezes?

Responda no chat, sem identificar

ABERTURA

Agenda do bloco

Hoje · 24/10

9h00	10h00	10h10 às 11h30	11h30
Exposição: revisão, segurança, dependências, dívida	**Pausa**	**AV1.5 · checklist construído por vocês e revisão de um PR de agente**	**Socialização e fechamento**

Disciplina · 6 blocos

B1 · 16/10	B2 · 17/10	B3 · 17/10	B4 · 23/10	B5 · agora	B6 · 24/10
Modos e ambiente	**Escopo e refatoração**	**Testes e docs**	**Legado**	**Revisão e segurança**	**Produtividade e time**

A PERGUNTA DO BLOCO

Quem aprova o merge responde pelo código

Independente de quem digitou. A pergunta do bloco: o que o revisor costuma deixar passar?

O QUE MUDA

O que muda no review quando ninguém pensou o código linha a linha

1. Sem autor para perguntar a intenção
2. Aparência polida reduz o ceticismo
3. Volume de PRs aumenta
4. O texto do PR é gerado pelo mesmo sistema que fez a mudança

VIÉS DE AUTOMAÇÃO

Tendência a confiar demais na saída de sistemas automatizados

Código limpo e bem formatado vira falso indício de correção. ("LGTM")

O QUE MUDA

Funil de revisão

1 Intenção e escopo

2 Testes primeiro

3 Contratos e superfície pública

4 Segurança e dependências

5 Estrutura e manutenção

6 Execução independente

VULNERABILIDADES

O que os estudos dizem sobre segurança

~40%

de 1.689 programas do Copilot eram vulneráveis, em 89 cenários

Pearce et al. (2021)

Menos seguro

Usuários de assistente escreveram código menos seguro e ficaram mais confiantes na segurança

Perry et al. (2022)

~45%

das tarefas com falha conhecida

Veracode (2025) · fonte secundária

RESSALVA Modelos antigos; use como padrão de falha, não como taxa atual.

VULNERABILIDADES

Famílias de falha mais frequentes

Injeção
entrada concatenada em SQL, shell ou template

Controle de acesso (IDOR)
autentica, mas não confere o dono do objeto

Path traversal
caminho de arquivo montado com entrada do usuário

SSRF
o servidor busca a URL que o usuário fornece

Segredos
chave ou senha no código ou no log

Criptografia fraca
algoritmo obsoleto, aleatório previsível

Desserialização insegura
dado externo carregado como objeto

Sessão e CORS
cookie sem flags, origem liberada com credenciais

Logs com PII
dado pessoal gravado em log

Race condition
checar e agir sem trava

DoS
entrada sem limite de tamanho ou de tempo

VULNERABILIDADES

Injeção SQL

Parece inofensivo e é a falha mais clássica.

VULNERÁVEL

```
@app.get("/relatorio")
def relatorio():
    cliente = request.args["cliente"]
    rows = db.execute(f"SELECT * FROM vendas WHERE cliente = '{cliente}'")   # VULNERÁVEL
    return jsonify(rows)
# CORREÇÃO: db.execute("SELECT * FROM vendas WHERE cliente = ?", (cliente,))
```

VULNERABILIDADES

IDOR: autentica, mas não autoriza o objeto

Qualquer usuário lê qualquer pedido.

VULNERÁVEL

```
@app.get("/pedidos/<int:pid>")
@login_required
def ver_pedido(pid):
    return jsonify(Pedido.query.get_or_404(pid).to_dict())     # VULNERÁVEL: qualquer usuário lê qualquer pedido
# CORREÇÃO: Pedido.query.filter_by(id=pid, usuario_id=current_user.id).first_or_404()
```

Path traversal

O nome vem do usuário e atravessa diretórios.

VULNERÁVEL

```
@app.get("/download")
def download():
    return send_file(os.path.join(UPLOAD_DIR, request.args["nome"]))   # VULNERÁVEL: ../../etc/passwd
# CORREÇÃO: send_from_directory(UPLOAD_DIR, nome) + validação do nome
```

FALSA SEGURANÇA

Falsa sensação de segurança

MITO	"Os testes passam."	REALIDADE	**Não significa que é seguro.**
MITO	"Os testes cobrem o caso."	REALIDADE	**Testes de autorização negativos quase nunca são gerados espontaneamente.**
MITO	"Outro modelo revisou."	REALIDADE	**Revisão pelo mesmo modelo tem erros correlacionados.**

FALSA SEGURANÇA

Controles independentes

Controle	Exemplos
SAST	Semgrep, CodeQL
SCA	OSV-Scanner, pip-audit, npm audit
Secret scanning	gitleaks
Testes de abuso	o usuário A tenta acessar o recurso de B
Modelagem de ameaças	para mudanças sensíveis

DEPENDÊNCIAS

Pacotes que não existem

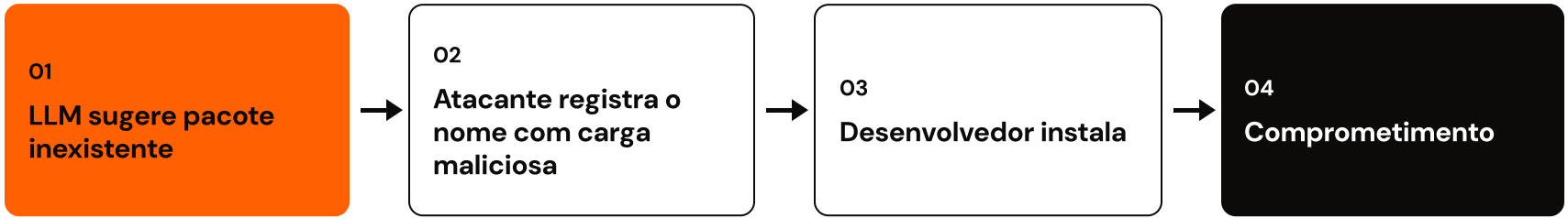
19,7%

dos pacotes recomendados não existiam

5,2%	em modelos comerciais
21,7%	em modelos abertos
205 mil+	nomes fabricados
576 mil	amostras de código, 16 modelos

TIPO Estudo empírico em larga escala.

Slopsquatting



DEPENDÊNCIAS

Checklist de dependência nova

- [ ] O pacote existe e é o certo?
- [ ] Precisa mesmo (a lib padrão resolve)?
- [ ] Vulnerabilidades conhecidas
- [ ] Scripts de instalação e transitivas
- [ ] Quem mantém, há quanto tempo?
- [ ] Versão fixada e lockfile
- [ ] Licença compatível
- [ ] Aprovação explícita no time

DEPENDÊNCIAS

Outros vetores com agentes

1. Injeção de prompt em README, issue, página web ou resposta de servidor MCP
2. Extensões de agente comprometidas
3. Arquivos de configuração e skills

REGRA Trate extensões, servidores MCP e skills como dependências.

Duplicação, acoplamento e inchaço

- [ ] Reinvenção de helper que já existe
- [ ] Acoplamento escondido
- [ ] Código morto
- [ ] Mudança de API sem aviso
- [ ] Cópia e cola em vez de abstração
- [ ] Defensividade excessiva
- [ ] Erro tratado de forma inconsistente
- [ ] Consultas N+1

Defesas automatizáveis

1. Regras de arquitetura executáveis (ArchUnit, import-linter, dependency-cruiser)
2. Detecção de clones em CI
3. Linters do projeto
4. Pergunta de revisão: "já existe algo que faz isso?"

Dívida que só aparece na manutenção

Dívida	Como aparece
De compreensão	Ninguém explica por quê
De verificação	Testes frágeis ou tautológicos
De documentação	Docs que não acompanham o código
De dependências	Pacotes sem dono ou sem atualização
De arquitetura	Padrão novo que ninguém decidiu

Como fazer a dívida aparecer cedo

1. Pergunta na revisão: quem do time explica e mantém isto?
2. ADR para novo padrão
3. Acompanhar retrabalho pós-merge
4. Rotacionar a propriedade

PR DE AGENTE

1. Intenção e escopo
2. Como foi verificado (comandos e saída)
3. Teste que falharia sem a mudança
4. Superfície afetada
5. Participação de IA
6. Responsável humano nomeado

PR DE AGENTE

Template de descrição de PR

O que o PR deve trazer.

```
## Intenção e escopo
O que muda e por quê. O que NÃO muda. Link do plano/ticket.
## Como foi verificado
Comandos executados e saída. Testes novos e o comportamento de cada um.
Evidência de que o teste falha sem a mudança.
## Superfície afetada
Contratos públicos, migrações, configuração, dependências novas (nome, versão, motivo).
## Riscos e pontos de atenção
## Participação de IA
Modo usado. O que a IA errou e o que foi corrigido à mão.
```

PR DE AGENTE

Gates mínimos

- [ ] Tamanho limitado
- [ ] Teste que falharia sem a mudança
- [ ] Sem segredos
- [ ] CI verde e execução independente pelo revisor
- [ ] Dependência nova com justificativa
- [ ] Edição de teste existente justificada

Janela: 10h10 às 11h30 Faltam 18:29

Construam o checklist

Plenária, 25 min

1. Cada um cita no chat um defeito que já viu
2. Agrupamos em categorias
3. Cada falha vira uma pergunta verificável
4. Marcamos o que bloqueia o merge

ONDE

`exercicios/aula-5/README.md`

`exercicios/aula-5/template-registro.md`

O PR é liberado pelo professor em `apoio/pr-bloco5/` na parte 2.

Janela: 10h10 às 11h30

Esqueleto de partida do checklist

Escopo	
Correção	
Testes	
Segurança	
Dependências	

Estrutura	
Desempenho e escala	
Manutenção	
Processo	

Eixo × pergunta de revisão. Preenchido ao vivo.

Revisem o PR do agente

1. Aplique o checklist e **rode você mesmo** o que a descrição do PR diz que rodou
2. Registre **ao menos 4 achados**: onde, impacto, severidade (bloqueia ou comenta), correção e o teste que falharia
3. Confira as alegações do texto do PR
4. Decida o merge

Tempo: 40 min

ONDE

`apoio/pr-bloco5/`, liberado agora

`exercicios/aula-5/README.md`

`exercicios/aula-5/template-registro.md`

Comentário de revisão: fraco x técnico

FRACO **"parece arriscado"**

TÉCNICO

```
BLOQUEIA MERGE: a rota autentica, mas não autoriza por objeto. O usuário A consegue
exportar dados do usuário B. Sugestão: filtrar por `tenant_id` e adicionar um teste negativo
("usuário A não acessa o recurso de B").
```

Problema, impacto e correção, com teste negativo.

Janela: 10h10 às 11h30

AV1.5 em salas: 40 minutos

Grupos	**4 a 5 pessoas**
PR	**dividido em partes, cruzem**
Transição de salas	**5 a 8 min**

VOLTEM ÀS

11h30

Faltam 18:29

CESAR

Que problema real quase passou despercebido no seu grupo?

3 a 5 grupos

RESPONSABILIDADE

Responsabilidade e conformidade

1. Responsabilidade técnica e profissional é de quem aprova
2. A responsabilidade jurídica depende de contrato, política da empresa e jurisdição
3. Dados em prompts têm implicações de proteção de dados (LGPD)
4. Sugestões podem se parecer com código de licença restritiva

AVISO Não é parecer jurídico.

FECHAMENTO

Quem aprova o merge responde pelo código, independente de quem digitou

Verifique.

Execute.

Exija teste que falha.

C E S A R

Até 30/10, 23h59

Entrega final: AV3

1. Um PR assistido
2. Registro do que a IA errou
3. Meia página de medição

No MiniLoja: B1 acesso a pedidos · B2 `Decimal` · B3 reserva de estoque

Leituras: Perry et al. (2022) · Pearce et al. (2021)

ONDE

avaliacoes/av3/README.md

avaliacoes/av3/template-registro.md

FECHAMENTO

Referências

Perry et al. (2022). Do users write more insecure code with AI assistants?
arXiv 2211.03622

Pearce et al. (2021). Asleep at the keyboard?
arXiv 2108.09293

Spracklen et al. (2025). USENIX Security.
arXiv 2406.10279

Veracode (2025). GenAI Code Security Report.
Fonte secundária

HOJE · 14H

Produtividade real, medição e uso em time

AV1.6 às 15h10

github.com/jamissonfreitas/repo_alunos_d12_esia

CESAR